SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

StockSage is an AI-powered Decision Intelligence system designed to empower Malaysian SMEs by optimizing inventory management. By integrating Z.AI's General Language Model (GLM), the system transforms fragmented inventory data into actionable economic strategies, such as dynamic pricing and flash sales, to prevent waste and maximize revenue.

1.1. Purpose:

The purpose of this document is to outline the technical architecture and design decisions behind the StockSage ecosystem.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *A modular structure with clear separation between the decision engine and data layers.*
2. ***Data Flows:***
    a. *The movement of inventory records from local storage through Z.AI for strategic reasoning.*
3. ***Model Process:***
    a. *The end-to-end workflow from detecting critical stock levels to generating AI-driven business recommendations.*

1.2. Background:

Many Malaysian SMEs face challenges in making timely decisions due to manual or intuition-based processes, often leading to significant losses from expired goods. StockSage addresses this by providing a context-aware 'SME Advisor' that utilizes data-driven insights to optimize economic outcomes.

1. *Previous Version:*
    a. *StockSage is an original build for UMHackathon 2026. There are no previous versions.*
2. *Changes in Major Architectural Components*
    a. *StockSage is an original build for UMHackathon 2026. There are no previous versions.*
3. *New capabilities that are only introduced in this*
    a. *StockSage is an original build for UMHackathon 2026. There are no previous versions.*

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
SME Owner	*Primary user managing inventory and pricing.*	*Actionable strategies to reduce waste and improve revenue.*
Development Team	*Build and maintain the StockSage platform.*	*Modular codebase, clear API contracts, and scalable architecture.*
QA Team	*Validating system behavior and Z.AI reasoning.*	*Accuracy in AI insights and system stability under varying data loads.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web-based Decision Dashboard*
Architecture	*Client-Server model using Spring Boot REST API*

StockSage is structured as a client-server system where the Frontend Dashboard communicates with a Spring Boot Backend. The Backend manages data persistence via Oracle Database 19c and facilitates communication with the Z.AI GLM service layer for strategic inference.

2.1.2 LLM as Service Layer

Z.AI GLM is integrated as the core reasoning engine rather than a generic module. The following Dependency diagram includes:

a. *Prompt Construction: The system extracts 'Critical' inventory data (e.g., items with < 3 days shelf life) and environmental context (e.g., weather forecasts) to build structured prompts.*
b. *Context Handling: Prompts include stock levels, expiry dates, and historical sales trends within the context window.*
c. *Response Parsing: The system parses Z.AI's JSON output to display specific 'Actions' (e.g., Flash Sale) and 'Reasoning' on the dashboard.*

2.1.3 Dependency Diagram

This section describes how data and commands move between the dashboard and the AI reasoning layer.

a. *Prompt Construction: The Backend extracts raw rows from the PRODUCTS table (ID, Name, Stock, Expiry Date) and injects them into a pre-defined system template: "You are an SME advisor. Analyze these items and provide a discount strategy...".*
b. *Context Window: The context includes the current system date, a list of critical inventory items, and a hardcoded weather/event scenario to simulate "context-aware" reasoning.*

© UMHackathon 2026

c. *Response Handling: The system receives a raw String/JSON from Z.AI GLM, parses the action and reason fields using a JSON provider, and maps them to a Java DTO (Data Transfer Object) for the frontend.*
d. *Token Management: Input is limited to the top 20 most critical items to ensure the prompt stays within the GLM-4 token limit and prevents API timeouts.*
e. *Major API Calls:*
    1. *Frontend → Backend: **GET /api/dashboard-data** to request current status.*
    2. *Backend → Z.AI API: **POST /v4/chat/completions** to perform strategic inference.*
    3. *Backend → Oracle DB: **SELECT** queries via JPA Repository to fetch inventory records.*

2.2.2. Sequence Diagram

a. *Step 1: The SME Owner opens the StockSage Dashboard (Frontend).*
b. *Step 2: The Dashboard triggers an automatic fetch call to the Spring Boot REST Controller.*
c. *Step 3: The Controller calls the AIService, which pulls 'Critical' stock data from the Oracle Database.*
d. *Step 4: AIService sends the inventory data to the Z.AI GLM-4 API.*
e. *Step 5: Z.AI returns a specific pricing strategy (e.g., '30% Flash Sale').*
f. *Step 6: The Dashboard updates the UI to show the strategy and reasoning to the owner.*

2.2. Technological Stack

2.2.1. *Frontend: HTML5, CSS3 (Inter Font), and Bootstrap 5.*
2.2.2. *Backend: Java 21, Spring Boot v4.0.5.*
2.2.3. *Database : Oracle Database 19c.*
2.2.4. *AI Engine: Z.AI GLM-4 API.*

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)

a. *Inventory data originates in the Oracle DB (Source), moves through the Spring Boot Logic (Process), is enhanced by Z.AI (External Entity), and is finally presented to the SME Owner (Destination).*

2.3.2. Normalized Database Schema

a. *The PRODUCTS table is designed with fields for PRODUCT_ID (Primary Key), STOCK, and EXPIRY_DATE to ensure data integrity and allow for efficient sorting of "Critical" items.*

© UMHackathon 2026

3. Functional Requirements & Scope

3.1. Minimum Viable Product:

3.1.1.

#	Features	Description
1	Inventory Health Monitoring	Real-time tracking of shelf life and stock quantity.
2	Z.AI Strategy Generator	Automated generation of pricing strategies for critical items.
3	Economic Impact Dashboard	Visualizing current stock value and potential savings.

3.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Scalability	Support for multiple SME store IDs.	Relational database schema designed for multi-tenant data isolation.
Reliability	99% uptime for the Decision Dashboard.	Fallback mechanisms to display basic inventory if Z.AI API is unreachable.
Maintainability	Clean separation of concerns.	Modular package structure: controller, service, model, and repository.
Token Latency	Z.AI Strategy generation < 3s.	Asynchronous API calls with loading states to ensure smooth UX.

3.3. Out of Scope / Future Enhancements

a. POS Integration: Direct execution of the AI strategy into a Point-of-Sale system to automate price changes.
b. Push Notifications: Sending real-time alerts to the owner's mobile device when Z.AI detects a critical risk.
c. Predictive Demand: Using historical sales data to predict exactly how many units will sell before expiry

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing: *For new Z.AI strategy templates, StockSage will implement a phased rollout. We will release updated reasoning models to an initial 5% of SME user accounts (canary release) to monitor API latency and the accuracy of generated pricing advice. If the 'Success Rate' of executed promotions remains high and no technical anomalies are detected, the rollout will gradually increase to 100% of the user base. Furthermore, to optimize economic impact, we will test different Z.AI prompt versions. For example, Group A receives 'Flash Sale' recommendations while Group B receives "Bundle Deal" recommendations for the same inventory conditions. The strategy that results in a higher "Stock Liquidation Rate" will be adopted as the primary decision logic.*

4.1.2. Emergency Rollback (ER) & Golden Release: *In the event of a critical failure—such as the Z.AI API returning malformed JSON or the Oracle Database reaching maximum connection capacity, an automated ER is triggered. This immediately halts current requests and redirects users to a stable state to prevent dashboard crashes. Furthermore, StockSage maintains a 'Golden Version' of the application that relies on local, rule-based decision logic instead of live AI inference. If the primary AI engine becomes unstable, the CI/CD pipeline automatically redeploys this Golden Version to ensure SME owners can still access their basic inventory health data without interruption*

4.1.3. Priority Matrix:

4.1.3.1. *P1 Critical: If the connection to Oracle DB fails, the system triggers a fallback to cached local data to maintain dashboard availability.*

4.2. Monitoring:

4.2.1. *Health Checks: The system uses Spring Boot Actuator to monitor the status of the server and the connection to the Oracle Database in real-time.*

4.3. Assumptions:

a. *The SME owner maintains an accurate inventory count within the Oracle Database.*
b. *The Z.AI API is accessible via a stable internet connection during business hours.*

4.4. External Dependencies:

Tools	Purpose	Risks
Z.AI GLM API	Core reasoning engine for economic strategies.	High: Product value fails if API is down; mitigated by graceful degradation.

© UMHackathon 2026

Oracle JDBC	*Persistent data storage for products/sales.*	*Medium: Connection errors; mitigated by DataSource error handling.*

5. Project Management & Team Contributions

5.1. Project Timeline:

1. *Day 1: Database schema design and Oracle DB setup.*
2. *Day 2: Backend Spring Boot integration and Z.AI API testing.*
3. *Day 3: Frontend Dashboard development and End-to-End integration.*
4. *Day 4: Final validation, documentation (SAD/QATD), and pitching video recording.*

5.2. Team Members Role:

1. *Farhah (Leader): Backend Architecture Lead – Spring Boot and Z.AI Integration.*
2. *Aleeya: Frontend Lead – UI/UX Design and API Fetch Logic.*
3. *Ain: Database Architect – Oracle SQL Schema and Data Integrity.*
4. *Hanah: QA & Documentation Lead – Quality Assurance and System Analysis*

© UMHackathon 2026